

Applied Machine Learning Optimization in Machine Learning



UNIVERSITY OF
GOTHENBURG

CHALMERS

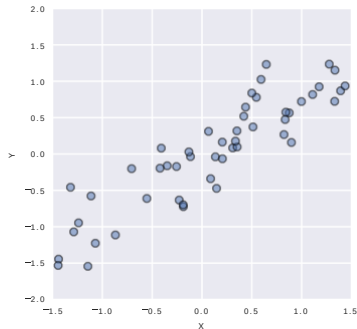
optimizing objective functions in machine learning

in many ML algorithms, we train a model by optimizing an
objective function

we set the parameters to the best values

example: linear regression, one variable

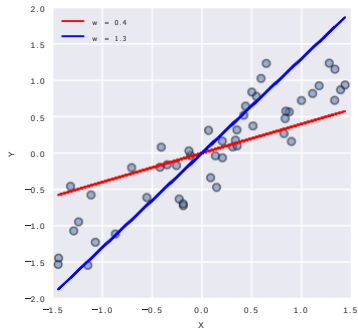
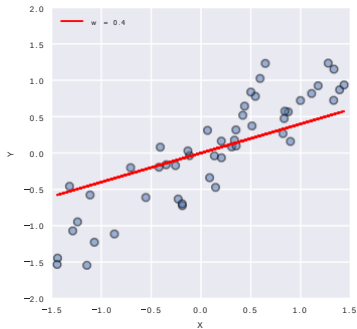
our data



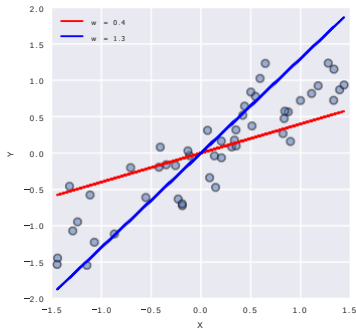
our model: $\hat{y} = w \cdot x$

so how do we find the “best”
value of the parameter w ?

what is a good value for w ?

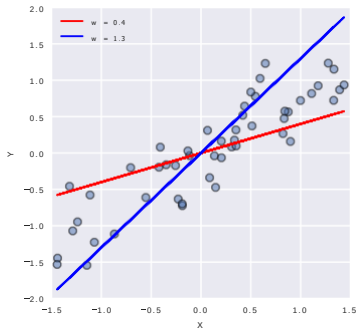


what is a good value for w ?



$$\text{MSE} = \frac{1}{n} \sum_{i=1}^N (y_i - w \cdot x_i)^2$$

what is a good value for w ?

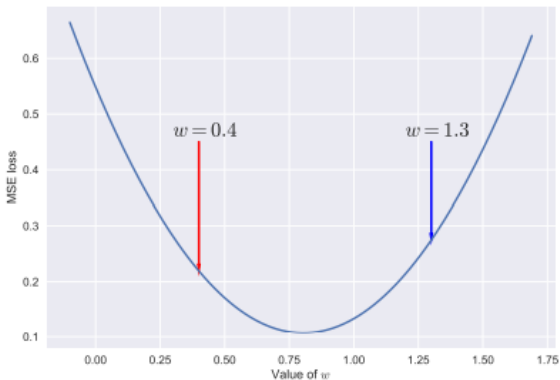


$$\text{MSE} = \frac{1}{n} \sum_{i=1}^N (y_i - w \cdot x_i)^2$$

for $w = 0.4$: MSE= 0.22

for $w = 1.3$: MSE= 0.27

minimizing MSE with respect to w

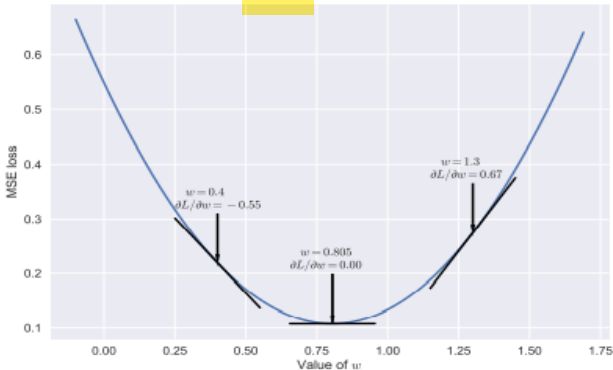


best value is around $w = 0.8$

how to find it **automatically**?

Derivative

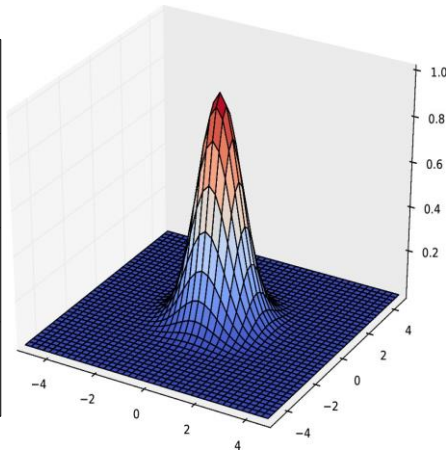
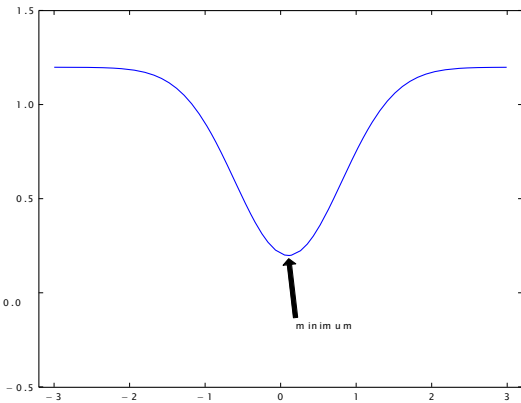
- the **derivative** of a function represents the **rate of change**
- graphical intuition: the **slope** of the curve



in realistic ML, we will have many parameters! (w is a vector)

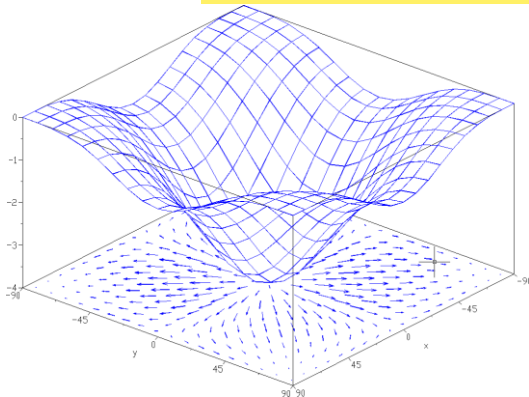
for a **single parameter w_i** , we can define the **partial derivative**

Examples



the gradient: graphical intuition

- the vector of all partial derivatives is called the **gradient**
- geometrical intuition: the gradient points in the uphill direction



- again: the gradient is zero if we have an optimum

Optimizing objective functions

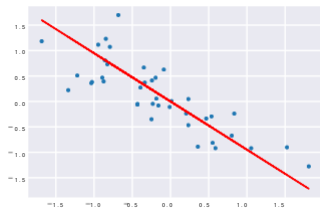
objective functions for machine learning

- a **loss function** that measures how well the model fits the training data
- a **regularizer** that measures how simple/complex the model is

minimizing squared errors

- in the **least squares** approach to linear regression, we set $\mathbf{w}$ to minimize the mean (or sum) of **squared errors**

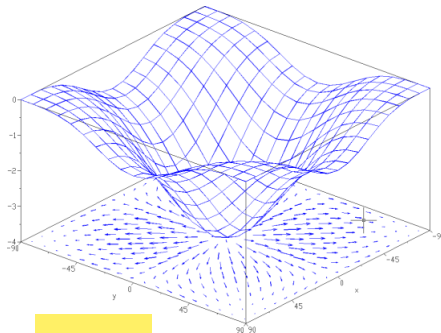
$$f(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (\mathbf{w} \cdot \mathbf{x}_i - y_i)^2$$



- how can we make this into an actual algorithm?

using the gradient

- geometrical intuition: the gradient points in the uphill direction



- To find the minimum:
 - take a small step in the direction opposite to the gradient
 - repeat until the gradient is close enough to zero
- this is called **gradient descent**

gradient descent, pseudocode

- the same thing again, in pseudocode:
 1. set $\mathbf{w}$ to some initial value, and select a suitable step size η
 2. compute the gradient $\nabla f(\mathbf{w})$
 3. if $\nabla f(\mathbf{w})$ is small enough, we are done
 4. otherwise, subtract $\eta \cdot \nabla f(\mathbf{w})$ from $\mathbf{w}$ and go back to step 2
- conversely, to find the maximum we can do **gradient ascent**:
then we instead add $\eta \cdot \nabla f(\mathbf{w})$ to $\mathbf{w}$

Discussion: step size

Step size:

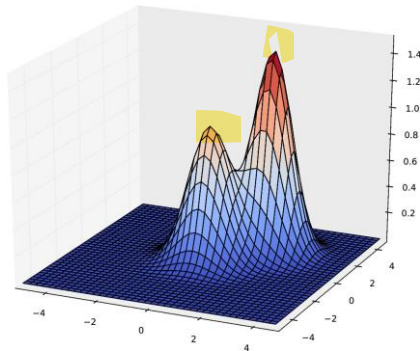
- Too large
- Too small

variable step size

- smarter versions of gradient ascent/descent try to **adapt** the step size so that we don't go too slow in the beginning, or bounce around the top at the end
- simple solution: start fast, then gradually more slow
- more about this when we discuss neural networks

local and global maxima/minima

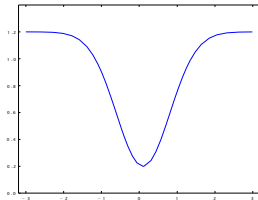
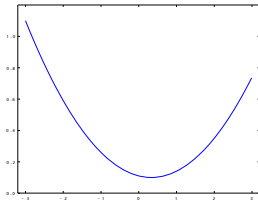
- some functions have **local maxima or minima**



- these functions are harder to optimize because the local (but not global) optima also have a gradient of 0

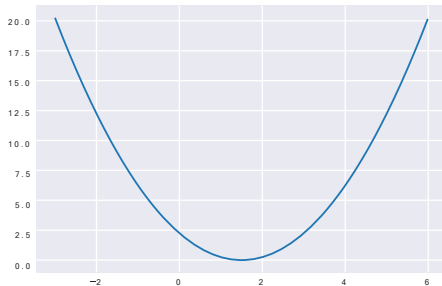
convex and concave functions

- a function is **convex** if it always curves downwards
- equivalently, if we draw a line between two points of the surface, the surface is always below the line



- the point of this: if we find a local minimum of a convex function, this is **guaranteed to be the minimum**

example: shape of the squared error loss



Gradient descent

- in machine learning, our objective functions are often defined as sums over the whole training set, such as

$$f(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (\mathbf{w} \cdot \mathbf{x}_i - y_i)^2$$

- in this case, it is cumbersome to compute the gradient
 - because it depends on all the instances in the training set

Gradient descent

Example:

- we have 10,000 data points and 10 features
 - We need to compute the derivative of this function with respect to each of the features: $10000 * 10 = 100,000$ computations per iteration.
data points * features
 - If we take 1000 iterations: $100,000 * 1000 = 100000000$ computations to complete the algorithm.
- Gradient descent can be slow and computationally very expensive to perform on huge data.

stochastic gradient descent

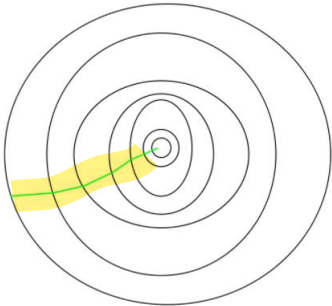
- **Solution: stochastic gradient descent:** simplify the computation by computing the gradient using just a small part
 - in the extreme case, a **single** training instance or
 - **minibatch:** a few instances

GD vs SGD

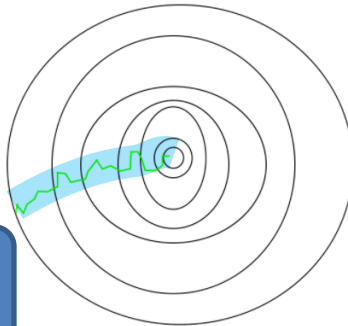
SGD is generally noisier than typical Gradient Descent

SGD usually took a higher number of iterations to reach the minima because of its randomness in its descent

The path took by Batch Gradient Descent -



A path has been taken by Stochastic Gradient Descent -



SGD still computationally much less expensive than typical Gradient Descent.

SGD: pseudocode

if we use all the training dataset we will stop when the gradient is close to

1. set w to some initial value, and select a suitable step size η
2. select a single training instance x
3. compute the gradient $\nabla f(w)$ using x only
4. if we are "done", stop
5. otherwise, subtract $\eta \cdot \nabla f(w)$ from w and go back to step 2

If we use a single instance this probably a bad idea because for some training instance, the Gradient will be close to 0 but for other training instance we might be very far off !

when to terminate SGD?

- simple solution: fixed number of iterations
- or evaluate on a held-out set: **early stopping**

In SGD, what is evaluate on held-out set: early stopping?

- Held-out set is a separate data set from the training data
- To make the computation of SGD faster, early stopping can be set to True, so that the training stops as soon as the performance on the evaluation set (i.e., the held-out set) does not improve. The same principle is used when setting early stopping in neural network.
- See also https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html#sklearn.linear_model.SGDClassifier

Notebook:

- Gradient ascent example
- GD and SGD regression example

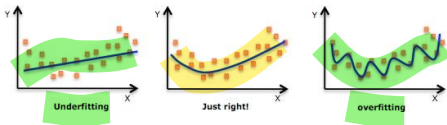
recap: main points

- much of ML involves **optimizing objective functions**
- a simple algorithm for optimization: **gradient descent**
- faster and simpler variation: **stochastic gradient descent**

Linear regression and regularization

recall: the fundamental tradeoff in machine learning

- **goodness of fit**: the learned model should describe the examples in the training data
- **regularization**: the model should be simple



[Image source]

what does it mean to “keep the model simple”?

- concretely, how can we add regularization to the linear regression model?

what does it mean to “keep the model simple”?

- concretely, how can we add regularization to the linear regression model?
- the most common approach is to add a term that **keeps the weights small**
- for instance, by penalizing the squared length (norm) of the weight vector should be small:

$$\|w\|^2 = w_1 \cdot w_1 + \dots + w_n \cdot w_n = w \cdot w$$

- this is called a **L_2 regularizer** (or a L_2 penalty)

combining the pieces

- we combine the loss and the regularizer:

$$\frac{1}{N} \sum_{i=1}^N \text{Loss}(\mathbf{w}, \mathbf{x}_i, y_i) + \alpha \cdot \text{Regularizer}(\mathbf{w})$$

$$= \frac{1}{N} \sum_{i=1}^N (\mathbf{w} \cdot \mathbf{x}_i - y_i)^2 + \alpha \cdot \|\mathbf{w}\|^2$$

- in this formula, α is a “tweaking” parameter that controls the tradeoff between loss and regularization

ridge regression

- the combination of least squares loss with L_2 regularization is called **ridge regression**

$$\frac{1}{N} \sum_{i=1}^N (\mathbf{w} \cdot \mathbf{x}_i - y_i)^2 + \alpha \cdot \|\mathbf{w}\|^2$$

- in scikit-learn: `sklearn.linear_model.Ridge`
- (unregularized least squares: `sklearn.linear_model.LinearRegression`)

discussion

$$\frac{1}{N} \sum_{i=1}^N (\mathbf{w} \cdot \mathbf{x}_i - y_i)^2 + \alpha \cdot \|\mathbf{w}\|^2$$

- what happens if we set α to 0?
- what happens if we set α to a huge number?

the Lasso and ElasticNet models

- another common regularizer is the L_1 norm

$$\|\mathbf{w}\|_1 = |w_1| + \dots + |w_n|$$

- the L_1 norm gives the Lasso model
- combination of L_1 and L_2 gives the Elastic Net model
- in scikit-learn:
 - `sklearn.linear_model.Lasso`
 - `sklearn.linear_model.ElasticNet`

Lasso gives sparse weight vectors

```
lr = LinearRegression()  
lr.fit(X, Y)  
lr.coef_
```

```
array([ 0.38118273, -1.78697508, -0.00315352, -0.00294201, -0.00720975])
```

```
ridge = Ridge()  
ridge.fit(X, Y)  
ridge.coef_
```

```
array([ 0.38085675, -1.78522631, -0.00302254, -0.00297011, -0.00715312])
```

```
lasso = Lasso(alpha=0.05)  
lasso.fit(X, Y)  
lasso.coef_
```

```
array([ 0.33035205, -1.73958828,  0.          , -0.          , -0.          ])
```

- this is an example of **embedded feature selection**

Regularized regression models notebook

Discussion:

Which of these following statement is true for regressions methods:

1. Ridge regression uses subset selection of features
2. Lasso regression uses subset selection of features
3. Both use subset selection of features
4. None of above

Discussion:

When compared with L1, L2 works well in case where we:

1. If we have more features.
2. If we have less features.
3. If features have high correlation.
4. If features have low correlation.

Explanation:

<https://zawar-ahmed.medium.com/comparing-linear-regression-models-lasso-vs-ridge-60587ff5a5aa>

Discussion: How to choose which regularizer to use?

- If we want to control the weights (keep the weights small, but not zero) and want to keep all features, use L2.
- If there are many features and we want to reduce the number of features then choose L1 (as it will reduce the weights of non-/less relevant features to zero).
- If we don't really care L1 or L2, we can use both (elastic net).

main points

- we can optimize objective functions for **linear regression** as for any other ML model
- **regularizers** can be introduced to mitigate overfitting
- the **Ridge** (L_2) and **Lasso** (L_1) regularizers penalize the vector w in different ways